



Securing a GraphQL API with Apollo 3

Course Overview

Course Overview

Hi everyone, my name is Mat Warger, and welcome to my course Securing a GraphQL API with Apollo. I'm a senior software consultant at Keyhole Software in Kansas City and have been building full-stack applications for over a decade. Security is an important part of any modern web application. Have you ever wondered what similarities or differences there are between GraphQL and traditional REST APIs? In this course, we're going to explore how API security practices can be applied to the Apollo GraphQL server in Node.js, and how to use the API securely from a client-side React app using Apollo Client. We'll leverage an existing application and slowly add features, including user accounts and roles, to provide a great experience for the users of the app in a secure way for the clients of the app to interact with it and the underlying API. Some of the things we'll cover include securing a GraphQL API using header authentication to allow for server and client-side requests, securing a GraphQL API using cookies to authenticate securely between a browser and our API, we'll implement role-based security so we can make sure that only certain users are allowed to access certain resources, and we'll protect against complex or malicious queries that could possibly bring down our server. By the end of this course, you'll have a solid foundation for securing your own GraphQL APIs and the concepts necessary to implement it in your own projects. Before beginning the course, you should be familiar with Apollo Server and Apollo Client. We'll be building up security features from scratch, so you won't need a security background to get started. I hope you'll join me on this journey to learn client and server security with GraphQL with the Securing a GraphQL API with Apollo course here at Pluralsight.

Implementing Header Authentication for Apollo Server and Client

Introduction

Hello. My name is Mat Warger and welcome to Securing a GraphQL, API with Apollo. Security is an important part of any modern web application, and in this module, we'll see how to secure and protect an existing Apollo API and Client application using headers. To lay the groundwork, we're going to look at the slick



redesign of the Globomantics website, where they've added a conference feature for an upcoming event. This feature allows anyone to view upcoming sessions and speakers. Sessions can be marked as favorites, and new sessions can be added. The rest of this module, we'll be implementing security for the API and updating features of the conference portion of the application. Authentication will allow us to restrict access to only specific queries or mutations. To accomplish this, we'll see how to add header authentication to the server and utilize that authentication in the client application. We'll start by creating a basic implementation of JSON Web Tokens for use in our headers. Then we'll see how to allow users to sign up for and sign into our application, so they can submit sessions and save their favorite sessions. Will also restrict access to portions of our API. We'll walk through it step by step, and learn all the necessary bits along the way. Now this may sound familiar to you if you've previously worked with other server-side APIs. Apollo uses a lot of the same techniques accomplished in slightly different ways. Let's get to it.

The Application

Let's talk about the libraries we'll be using. Our application is composed of an API driven by Apollo Server, a library for building GraphQL servers with Node.js. We'll also be working with a React app that uses Apollo Client to issue GraphQL queries and mutations to our server. Note that it will be difficult to continue with this course if you are not already familiar with Apollo Server and Apollo Client. If you need to get up to speed, check out the courses for Building a GraphQL API with Apollo Server and Consuming a GraphQL API with Apollo Client in React here on Pluralsight. With that out of the way, let's see what we're working with. Behold, a glimpse into our Utopian cyberpunk future. The Globomantics corporation wants to bring the brightest minds from all around the world together and hold a conference, so that knowledge can be shared with everyone. To help us all assimilate, I mean bring their vision to life, they have built a sleek conference feature on their website consisting of a React application that uses Apollo Client to communicate with the GraphQL API utilizing Apollo Server. When you visit the conference section, you can check out who is speaking. You can view the list of sessions, you can mark sessions as favorite, and you can submit new sessions. Globomantics has asked that we bring some modern security practices to their site to allow for authentication and authorization, as well as preventing any malicious activity. We can start by implementing a user sign-in and sign-up feature that we'll utilize HTTP headers and JSON Web Tokens. Next, let's take a look at some code. Here's the project that we'll be working with. We have an API directory that holds the server implementation and an app directory that holds the React client application. Our server entry file has the initialization for Apollo Server. We have a schema file that contains types for the speaker, the user, and the session. These are linked to resolvers, which are linked to data sources in this example Session, which holds



functions for operating on our data for the appropriate queries and mutations. On the app side, we have a fairly basic React application that contains a number of routes. The main sections we'll be focused on are the Conference section and the Auth section. The Conference has pages for viewing sessions, speakers, details for individual sessions and speakers, as well as a form to add new sessions. We'll be revisiting these as we add features and functionality to the application.

JSON Web Tokens

Let's talk about JSON web tokens, commonly referred to as JWTs, or JOTs. At a very basic level, they are strings that represent JSON data. These will be how we communicate information between the server and the client in a secure way. So how do they accomplish this? The format of a JWT is a string of data that, by virtue of a secret key, allows for information to be encoded and its integrity verified. The token is created on the server usually when a user signs in or creates an account and is passed to the client application for future use, the process is fairly straightforward. First, the server signs or verifies the token, this is done using readily available libraries. The token is attached to the response of a request and the client application stores it and attaches it to subsequent requests. In our case, we'll be starting out by storing it in local storage in our browser so that our React app can use it with Apollo client. So why JWTs? One of the surface level advantages is that there are many available libraries and implementations that make working with JWT simple. They're easily passed between clients and servers, so they're especially portable and don't take much space. You may be thinking that sounds cool, but why would we use JWTs instead of just passing strings or other data back and forth? Wouldn't simple strings be easier to deal with and take even less bandwidth? What makes them special? Let's find out and take a look at what JWTs are made of. Here is a JWT in all its glory. The components of the JWT are relatively straightforward. The first section contains the header. This is going to be the Base64 representation of typically the type and the algorithm used for generating the token. The second section of the JWT is going to contain the part that we will most care about, that is the claims. Claims are meant to represent statements about the user whose data we've encoded. Think of these as properties like a name or email or maybe a flag like whether or not the user is an admin. The final section is the signature. This is created by combining the first two sections of the token with a secret string that you control. The string is what allows one to verify JWT because that string is the only way to decode a token back into data. Speaking of decoding, let's break down these sections a little further. Here, we see each section of our token and the decoded payload each section represents. The JSON structures for the header and payload are Base64 encoded to produce the resulting string. The header contains the algorithm used to encode the token and the type of token. The payload contains a number of claims about the user. In this case, we have a sub value which stands for subject. You can think of this as an ID



for the user. We also have the name, you can store any arbitrary data here, but note that the more data you store, the larger the resulting payload string will become when encoded. The final section is the signature which is the result of passing the Base64 encoded values for the header and the payload to an encoding function along with a secret string. This last part is important. You should keep the secret safe as it is the one way to ensure the integrity of your data. You should not keep it in source control or use it directly in a client application. If someone else were to get a hold of this secret, they could craft JWTs that would be valid when verified by your application. Let's see what this means by heading to jwt.io and seeing a token in action. This is jwt.io. This site allows you to play with JWTs using an interactive debugging tool. On the left is a text area where you can input a token and have the decoded payload shown on the right. Alternatively, you can change the payload on the right and watch the token update on the left. You'll see that the payloads from the token match, and if I change any of the values, the token also changes. The key part of this, for the sake of ensuring the integrity of our data, is the secret. Right now, we have a secret value of `secret123`. We can see in the bottom left that the signature is verified. This means that the combination of the header payload in the generated signature that are part of the JWT string on the left match the output of the algorithm ran, in this case, HMACSHA256 on the right. Because the signature is what verifies the integrity of the data, let's see what happens when we mess with some of this data. If I modify the signature by adding a character, for example, we can see that this results in an invalid signature. This is true because that generated output no longer matches. What about our data integrity? Recall that the middle section refers to our payload. In this case, a sub of `123456789` and a name of John Doe. If we replace this string with another that contains different data, maybe by attempting to add an admin role to our user, we can see our invalid signature result again. The payload no longer matches the encoded value that is in our signature. In this way, we can ensure the value that we send is verified against the signature with our secret key, and we can prevent the altering of a token to inject changes to the values that we may not want. One last important thing to note is that this only ensures integrity, it does not ensure secrecy. We are not encrypting these values, only encoding them. If you want to keep the claims a secret, there are ways to accomplish that, but that's out of the scope for now. If you want to learn more about JWTs or how to use them effectively, you can visit the website we just looked at, jwt.io.

Implementing User Sign-up and Sign-in in Apollo Server

Alright, let's get coding. For our demo, we're going to look at implementing our user sign-up and sign-in feature for our app. For signing up a user, we'll see how to use a mutation to pass user credentials, including a password which we'll hash in Salt for security and then store with the user data in our database. Next, we'll see how to allow a user to sign in by verifying their password and retrieving user



data from our database. In the process of adding these features, we'll implement basic security using JSON Web Tokens, including how to sign a payload to create a JWT, and verify a token to decode it to be sure the signature is valid. Finally, we'll use our newly minted authentication mechanism to restrict access to session submission mutations. So let's start on the server, first by implementing our sign-up feature. We'll first take a look at our schema. We'll see the `signUp` and `signIn` mutations, both of which take a `credentials` object that holds an email and password. They both return the `AuthPayload`. This is going to be the token that we get back, as well as some user information like the id and the email. Let's take a look at our mutations resolver. Here we can see a couple stubs for `signUp` and `signIn`. To begin with, let's think about how the sign-up process is going to work. A user will pass an email and password to our API, so those values will be coming over in the arguments as the `credentials` input. So let's destructure that. That `credentials` input is going to have the email and password in it, so let's destructure those as well. For good measure, let's also create a sanitized version of the input. Now the first thing we can do is check to see if the email address is already in use. We're using a basic data source for connecting to our db. We have multiple operations that we can perform on our users. We'll be focusing on the `getUserByEmail` to start. The `dataSource` is stored in context, so let's destructure that. Next, we'll check if the user already exists in the database, and if they do, we'll throw an error to let the client know. Now that we have valid user credentials, we need to create an account for the user in our database, but we don't want to store these without first encrypting the password. We'll use the utility function for doing this in the `authUtils` file. Let's import and call `hashPassword`. The hash function takes a password and we'll salt it by default to provide a defense against decryption. Now we can store our user credentials. We'll use the `user dataSource` for this to create our user. This will return the user that's been created. Now we'll use another helper function to encode a payload and get a token. The `createToken` function takes an object and uses the JSON Web Token library to sign the payload, along with our `SECRET` key. You may be wondering where that `SECRET` came from. We're using the `.env` library to allow us to have environment variables available in our application. At the root of the project, I have a `.env` file that holds my `SECRET` key. Note that your `SECRET` key in a real-world scenario should probably be generated by a computer. We require the config at the top of the server file, and that allows us to use the environment variables in our files. Now back to our resolver. We can use our `createToken` function by calling it with the user object we got back from the database. The final step is to return the user and token info from the resolver to match the return type that we established in the schema. This will be an object containing the token value and the user's email and id. Alright, let's start the application and try it out. So now let's try out that `signUp` mutation. We call the `signUp` mutation with some fake credentials, an email and a password. We'll ask for the token back, as well as the user and the id and email fields that we return. So let's run it and see what we get. There we go, we have our token, we have our user with the id and the email. It looks like everything worked. We can



further verify this by taking the token value and heading back to jwt.io to give it a try. On jwt.io, we can paste in the token value. Sure enough, there's our properties, the email and the sub. Right now our signature is invalid, but if we paste our secret into the verify signature box and repaste our token, we can see that the signature has been verified. Next up is `signIn`. This will follow a lot of the same patterns as before, so we'll start by copying the destructuring for the credentials and the `dataSources`. We'll also want to destructure the email and password like before, as well as sanitize our input, so we'll copy that down as well. We'll also make a couple changes we want to look for if the existing user doesn't exist, as well as returning a different error message. The next step is validating our password. We have a utility function for this called `verifyPassword`, so we'll utilize that to check if the hash that we looked up from the database matches the password that was passed in by the user. If it doesn't match, then we also want to return an error to the user. Now that our user is validated and the password is validated, it's just time to create a token and return the payload. This is the same as before, so we'll copy it from above. Alright, let's try it out. Here we are with our `signUp` mutation from before, and to change it to `signIn` we just need to alter a couple things. We'll go ahead and rename it, as well as changing the actual mutation to call `signIn`. It takes the same credentials object, so we should be good. And there we go, same payload comes back, so we get our token, our id, and our email. We can also check to make sure our errors worked. We change the email address to one that doesn't exist, and there we are, correct error message. Next, let's try to alter the password and make sure that our password verification is working. Looks good.

Implementing the User Sign-up and Sign-in Mutations

Next, let's take a look of the client-side application. Back in the Auth section of our site, we have a Sign-up form and a Sign-in form, so let's get these implemented. Back in the Auth section of the application, we first need to import a couple things from Apollo Client. We need the `useMutation` hook and the GQL function to create our GraphQL document and perform our mutation. Next, we'll implement the mutation. Next, we can use the mutation in our component. We'll call this function in the `handleSubmit`. Finally, we'll return the result value in a log, so that we can see it. Now that that's done, let's try it out. Alright, so let's put an email address and password, and give this a try. Alright, there it is, the sign-up data with our token, and our email, and ID. Now back to the application to do the sign in. So back in the application, the sign-in form is going to be nearly identical to the sign-up form, except for the names. We'll just copy some stuff over. Same goes with the mutation. Alright, good to go. Alright, back in the application, let's try to sign in with the user account that we just created. Once again, we can see the data that we get back is what we expect.



Authenticated Requests with Apollo Client

Back on the client side, let's take a look at the ApolloProvider. This is where we're creating our Apollo Client. If we want to make authenticated requests, we will need to modify this to add the appropriate header. But first, we need to have a way to hold the token information in our application. Let's do that by creating an `authContext`. We'll make a new file in the context folder. We'll call it `AuthProvider`. I'll paste in a basic provider here. This will hold the `authInfo` that we want, including the token and the user data. We'll also have a small function to determine if we're authenticated or not. This will come in handy later. And place these in the context value, so that we can use them in our application. The first step is to take the result of our `signUp` and `sign in` mutations and put them in our `authContext`. We'll navigate to our `Auth` component. In our `Auth` component, let's first import the `authContext`. Next, we can use this context in each of our components to hold the user data and token we get back from the server. First, we'll do `signUp`. We'll use the `useContext` hook from React to grab that context. Next, we'll get rid of the `log` and `destructure`, the appropriate values from the result of the mutation. Next, we'll assign that result using the `setAuthInfo` function from context. Next, we'll do `signIn`. We'll copy the same result down and change the names. Now let's get back to our `ApolloProvider`. Just as before, we'll import the `authContext`. That way, we can use it in the `useContext` hook. Now that we have the context value, we can grab the token off the `authInfo` object on our context. The HTTP link within our Apollo Client will allow us to add headers, so we will conditionally add an authorization header containing our token. The final step will be to surround our application in the `AuthProvider`. Back in the application, let's go ahead and sign in. We can first see that we got our data back, as we'd expect. If we go to check out the conference list, and we view the requests for sessions, we can see in the request headers that an authorization token Header has been added. Next up, we'll learn how to apply this token and use it on the server

Authentication Using the Apollo Server Context

Let's see how to authenticate using our token in the Apollo Server context. Now it's time to focus on our server entry file. The `ApolloServer` constructor currently takes arguments for the type definitions from our schema, the resolvers for our data, and the datasources that we're using to operate on our database. It can also take a context value, which we'll be using here. The context allows us to declare a function that will be ran on each request. We'll be able to inspect the request object to grab the authorization header and verify our token, so let's do that. We'll first declare our context as a function and destructure the request object. We'll be getting the information about our user based on whether a token is passed in, so we'll set up a variable to hold that user information. The next step is



inspecting our headers. If our request authorization header exists, then we'll want to verify the token. We have a utility function for this as well that uses the JSON Web Token library's verify function. If the token is verified as ours, we'll get the decoded value back. Once we have that value, we can assign it to the user object. Recall that this will be the decoded token value, which contains the subject, our user id, as well as the email for the user. To verify that this works, let's log out this payload and attempt to make a request using an authorization token. We'll run the `signIn` mutation. We can grab the token, and we can put the token in the authorization header for the request. Next, we can try to request something like sessions, and that worked. So let's see what our log showed us. There we go, the sub and the email of our logged-in user. Now that we have our user as part of the context, let's try and restrict the user from creating a session unless they're logged in. We'll open up the mutations resolver, and in this case we already know that the third argument is the context, and we have a `dataSources` property that's already being destructured. We've added our user to it, so now we can destructure user as well. At this point, if we have a user, we want to go ahead and allow this to continue, but if we don't have a user, we can go ahead and return null. We could also throw an error or return any other value that would be appropriate in this case. So let's give this a try in the app. We're back in the application, so now let's try to create a session. We can go to the Conference, Sessions page, and try and submit a session. For the title, let's go with GraphQL is Great! The description, It really is. Format will be, well, Session, and the level will be Beginner. We'll open up the Network tab to see if we can spot the result. If we hit Submit and inspect our request, we can see that it succeeded, and our `createSession` came back as null. We aren't currently logged in, so that's good. Let's log in and see it work. Once we sign in, we can go back to Sessions and click Submit a Session, and try to submit again. That time we end up getting a successful submission. We can also see the request header that holds our token with our logged-in user. It worked!

Summary

Alright, we have our Apollo Server API working with our React app using JSON Web Tokens, our users can sign up and sign into our app, and we've wired up the authorization header to send tokens to our API. If you intend for your API to be consumed by other services and not necessarily by an application like ours, header authentication might be all you need; however, there are some slight security issues regarding how to store these things on the client. We're going to leverage cookies to do this in the next module, allowing us to more securely store our tokens to persist across page refreshes and let us do less manual token manipulation. See you there.

Implementing Cookie Authentication for Apollo



Server and Client

Introduction

Alright, we've made some good progress so far with our application. We've made sure that no random person can walk up and just submit anything they want. They need to create an account and sign in. This is good. However, if they come back to the site or refresh the page for any reason, they have to sign in again. This is bad. We're going to look at using cookies to fix this issue and lean on the browser for storing our JWT. We'll have to make a few changes to the server to store this state. Along the way, we'll see how to use the identity of the user to save and retrieve their favorite sessions. To start off this module, we'll first take a brief look at cookies, what they are, why they're useful for storing session state, as well as the inherent trade offs compared with our current header authentication approach. Next, we'll get back to our authentication work and learn about storing, retrieving, and removing cookies to keep the user session intact. We'll make changes to the server to allow this by leveraging its express based nature. Finally, we'll learn how you would go about using the user's identity to fetch a user's favorite sessions, rather than merely restricting or allowing certain actions.

What Are Cookies?

Let's talk about the basics of cookies. A cookie is a piece of data that is used to communicate information from a server to a web browser. This can be any arbitrary data communicated as key value pairs. For this module, we'll continue using our JWT and see how to set it as a cookie. Cookies are controlled by the server. Once a response from the server is sent back with a cookie, the browser automatically stores it. If you need to remove a cookie, a request needs to be made to the server, so the cookie can be cleared. They are sent automatically by the browser with each request. This is built into the browser, so we can leverage this to always have our token available, including after refreshing. Before we implement this in our application, let's take a look at the uses and trade offs of cookies compared with headers, so we can better understand our path forward. When comparing cookies and headers, there are things to keep in mind that should help determine which might be a better fit for your application. The first commonly-cited difference between cookies and headers is that cookies can be stateful, which is the case when paired with a server that is storing a user's session. This is in contrast to headers, which are generally stateless, as was the case when using our JWTs in the previous module. We won't need to worry much about this distinction for this course, as we'll be using an otherwise stateless token, our JWT, as the content of our cookie out of simplicity. The next key difference between the two is that cookies are only viable in the browser. They're



meant to communicate information between a server and browser clients. Headers can be used in a browser, as we did, or passed to other servers as well. This makes them great if you need an API that can be called by other APIs or services. After authenticating, you can attach a cookie to a response. We'll see how to do this shortly. Similarly to how we can return a token value in the response, but with cookies, a browser will persist them automatically, so they can be used in subsequent requests and across page refreshes. Contrast that with headers and the need to attach a token each request. If you don't manually persist the token, a user will have to sign into your application every time they come back to it or refresh the page. If you do persist them, you need to store them off somewhere, and maybe you consider something like local storage, which is not a good place for JWTs. Once a cookie has been set, it is automatically attached to future requests by the browser, unlike headers, which must be manually set up so they send the appropriate value when requests are made. As I stated previously, cookies are controlled by the server, such that if you need to remove a cookie, you need to send a request to the server for it to do so. To make authenticated requests using headers, you merely remove the header in the client application. The last important difference comes down to vulnerabilities, and that cookies are vulnerable to cross-site request forgery, whereas headers are vulnerable to cross-site scripting attacks, with the latter being a larger surface area. We won't be digging into these in this course, but know that in this case you pick your poison. We're going to continue forward in this module with cookies, as we only need to serve and authentic our API with our own application, and we can leverage the browser's built-in functionality to do so. There are multiple ways to go about this, so the goal of this module is to get you started with a solid foundation.

Adding Cookie Support to Apollo Server

Let's get back to the code and implement cookies. When we send a request from the client containing a cookie, we'll need to extract it from the request so we can get our token. We could do this manually, but there's a better way. Apollo Server is based on Apollo Server Express, which is in turn based on Express. Because of this, we can use readily-available Express middleware to parse cookies from the request, so we can use them in our context. We need to make a few changes to our server file. Once we get these in place, we'll have an Express Server that is using Apollo to handle GraphQL requests. Let's start by installing cookie-parser. Next, we'll require it. Now we need to use it. For this, we'll make even more changes to our imports and server initialization. We can start by changing the aollo-server import from ApolloServer to apollo-server-express. Next, we'll require Express itself, and initialize an Express application. We'll follow convention and call it app. Next, above the apollo-server, constructor we'll add our first piece of middleware to handle cookies. Well, look at the effects of this in a minute, but for right now, we need to have the app use the middleware, so we can type in



app.use and pass in the cookie-parser. If you're following along, make sure you call the cookie-parser function. Finally, we'll need to apply our app middleware using our server to wire things together. Near the bottom of the file, we can say `server.applyMiddleware`, and pass an object containing our app. We'll then update our app initialization bit to have the app listen instead of the server, and modify the function accordingly. We'll need to be able to intercept cookies for each request. We've already done similarly for our headers, so we'll follow a similar pattern. Our cookie-parser middleware that is called above has run by this point, and its job is to parse the cookies that come over on each request, and attach them to the request, so we can use them. We'll change our token checks to pull from `request.cookies` instead of `request.headers`. Our verification can stay the same. So now we are pulling our cookies off the requests, just like we were pulling our token off the request. The other side of this, of course, is how we add our cookie with the token, just like how we added our header with the token. The main difference is that with cookies, the data you want to return to the client is attached to the response object itself, not in the data returned from any single request. To accomplish this, we'll need to pass the response to our resolvers, so we can use it for our `signUp` and `signIn` functions. We can do that by attaching to the context just as we did with our user. We'll destructure it and return it as part of our object. For our `signUp`, we'll need to first destructure our newly-added response object from the context argument. We can create our cookie by setting it using the cookie function on the response. We'll call it token, just like before, and add our token value. The third argument is an object that takes a number of options. The full list of these is available in the Express documentation. For now, we'll only set one, `httpOnly`. This flags the cookie to only be accessible by the web server. In other words, it's not accessible to JavaScript. This ensures that a malicious actor cannot grab our cookie using a script. Right at the end, we'll remove the token from the response. Next up is `signIn`, and we need to do the same thing as `signUp`, destructuring the response, and then we'll copy the cookie call that we just used, and remove the token from the response.

Adding Cookie Support to Apollo Client

Now we'll move on to the Client to use our newly implemented cookies. Since our server is now set up to accept and return cookies, we need to make a few changes. The first place we need to look is in the `ApolloProvider`. Recall that we were using the token provided by the `AuthProvider` to create a header. We no longer need that, so we can remove it. Next, we'll set the one thing we need, in our case, to make sure the cookies are sent with each request, and that's to specify our credentials as `same-origin`. This setting tells Apollo Client that we're going to send a cookie to the same origin, in this case, our localhost. Once this is done, we need to go back through the app and remove the references to token. We'll start in the `AuthProvider`, and while we're there, we'll also change the



isAuthenticated flag to look at our userData property instead. We'll clean up the usage of the token in the signUp form component, and also in the signIn component. Now that that's done, let's see what it looks like in the app. Alright, so we're back in the app with the developer tools open. Let's check on our cookie implementation by creating a new account. We'll go to the Sign Up page and type in an email and password. After we hit submit, we can check the browser dev tools and look at the Application tab. If we view our cookies, we should see our token value. This was sent back on the response after we successfully signed up, and was automatically stored by the browser. Now let's check our signIn function. We want to make sure that this works, so we'll clear out the cookie and refresh our browser. Next, we'll type in the email and password we just used for the new account, and hit Submit. We'll see the token is back again, and that it's sent back on the response just like the signUp function. Great, everything works, however, once you sign in, there's not a way to sign out. Our button is there, but it doesn't do anything at the moment. The component driving it is using the isAuthenticated Boolean flag in the AuthProvider to determine whether to show the Sign-In or Sign-Out buttons. If we refresh the page, we'll see the app changes to show signed out. But our token is still there. Since we have the token, we need a way to grab the user's information when we start the app so we can show the appropriate state. We'll tackle both of these issues next.

Handling Refresh and User Sign-out

We're going to knock out handling refresh and user sign out, so let's get right back into the code. Refreshing the app can be thought of as initializing the app in the first place so we can use that logic to think about what we need to accomplish. When the app starts up, we might already have a cookie stored in our browser. Because our Apollo client is set up to send that cookie, we can make a request to retrieve the available information, in our case, user data, based on the information stored in the token. We can start by making a mutation to handle this. We'll call it user info, and it will return the same AuthPayload we had before minus the token. Now we can create the resolver in our mutations file. The resolver will grab the user off of context. If we have a valid token, then the user will exist and would have been put on a context using our function that's part of the Apollo server initialization. In that case, we will return the user sub as the ID and the user's email. In the case that we either don't have a token or it's not valid, the user won't exist and will return an undefined user. Easy enough. Let's wire it up on the client. We'll need a way to call this mutation when the application starts. Let's create a new file called useappinit that will encapsulate the logic for getting our user info and setting it on our auth context. I'll paste one in here and we'll break it down. We can start at the top with the mutation. It grabs the user info that we just created on the server. Next, we have the useMutation hook which we'll use to fetch our data when the app starts up. We'll also pull in our auth context so we can set the user info once the mutation resolves. We return



the loading state so we can let the user know when the app is initializing, we'll use it in the app component. In `app.jsx`, we simply need to import this and call it at the top. We'll destructure our loading indicator and use it in a ternary so we can handle the state. Now let's check it out in the app. Now that we're back in the app, let's refresh the page and see it in action. Did you catch it? We can briefly spot the loading text and then see our header showing the sign up text, which means that our app called our server with the stored token cookie and retrieved the user info. We can also confirm this in the Network tab of our DevTools. So now we're signed in and we'll stay that way, but we need a way for the user to sign out, so let's handle that next. For user sign up, let's start out on the client this time. We need to add the functionality to our signout link. We can start with the mutation at the top. We'll create this on the server in a minute. Next, we'll use the `useMutation` hook to create the mutation and create the function that will perform the mutation. Once it completes, we'll set our user data to undefined. While we're here, we'll also use React router to the Sign in page once they're signed out. The last step is to attach our `onClick` handler to the link component. Now onto the server-side. We'll first start off in the schema by defining our mutation, it's called `signOut` and it returns an `AuthPayload`. Next, we'll head to the resolver's file and create the `signOut` resolver. Here is where we need to clear the cookie. We'll destructure the response from our context, and then we'll use it to call `clear cookie` and pass the name, in our case, `token`. This will tell the client to clear the cookie when the request completes. We can then return our undefined user. Now that we're done with that, let's head back to our app and try signing out. Back in the app, we should still be signed in. Let's fix that. We'll click the Sign out button and give the DevTools a look. The Sign in button is showing, we're redirected to the Sign in page, and we no longer have our token cookie. So there we go. We've gotten all the user stuff out of the way. Let's now use the fact that we know who the user is to handle letting them save their favorite sessions, that's what authentication is for after.

Using Identity to Save Favorite Sessions

We've got our user management handled. Now let's use the identity of the user to let anyone save their favorite sessions. The Sessions page contains the list of sessions with buttons on each one to let you toggle your favorites. It works at a really basic level, currently. Sessions themselves are marked as favorites. Now that our authentication is in place, we can depend on knowing who the user is and when they're signed in. We'll take advantage of this and create a way to save the users favorite sessions to their account. We'll start off by going through the app and removing the old session logic. The first part is at the data source level inside our sessions file. The next piece is the schema where we'll remove the favorite Boolean, and in the sessions file on the client-side, we need to remove it from the requested data in the queries. Now we need to update the current behavior for a toggle in the session to be tied to a user account. We want to store



the list of favorite sessions for our user so that they can add and remove them whenever they want. Our authentication gives us all the info we need to know for who is toggling the session, so we'll use that in our resolver. We'll start by updating the toggle favorite session mutation to return a user. We then need to update the mutations file to use the user data source and a new function that we'll add shortly. Because we're using the user identity for this, we should also check that it exists. It will take the session ID and we'll also pass the user ID so that we can use it in our database code. Next, I'll paste in the new entry to our user resolver. It looks up the value of the favorites for our current user and adds or removes the session based on whether it was currently a favorite. We then save the result and return the user. To fire this mutation on the client, we only need to modify a few things. The mutation name will be the same, but we'll need to update the sub fields to return the favorites array and grab the ID for each one. We'll also wrap the button and our isAuthenticated check so we can only show it when we have a valid user. We're already checking this on the server, so we have all our bases covered. We could try this now, but we have one issue, and that is with the favorites themselves. When we toggle one, we're writing the ID for a session to the database, but when we read a user and want to select their favorites, we need to have a resolver to grab the appropriate sessions based on those IDs and return the data. We'll accomplish this by adding a new resolver to our project for the user type. We should first add an entry file in the schema for the user type called Favorites, which will return a list of sessions. For the resolver, we can create a new file called users.js and export a favorites field resolver. The parent of this is the user and the data sources will be destructured from the context so we can get the favorites from our database. We'll retrieve our list using the user data source by passing the ID of the user. For each favorite in the list, we'll use our session data store to pull the information and add it to our list. We'll then return them from our resolver. We then need to add this resolver to our resolver's objects so it can be used. We'll require our file and add it, along with the others. To complete our implementation, we need to fetch whatever the signed in user's current favorites are when we query for the sessions list so we can highlight the appropriate star icons. We can use our authentication to provide any query that will get whatever the current user is. We'll call it me and add it to our query type in the schema. We'll create a quick resolver for this in our query.js file and simply grab the user that's passed in by its ID. If the user doesn't exist, we'll return undefined. Next, we'll head to the session.jsx file and add it to the queries for our session list and session detail. We'll grab the user with their ID, as well as the favorites and their IDs, which will be the ID of the sessions marked as favorites. Finally, we'll update our session item component usage. The session details component that's used in our list in the session detail view expect there to be a favorite Boolean flag. We'll use our return data to map over the list of favorites and return true if the current session ID is in the list. We'll also add this to the session detail component. So that was a good chunk of changes. We can check it out in the app and see how it works. So now we're back in the app with the sessions list. Let's see which ones we like and mark some favorites. AI-



powered robot sounds cool. Here is a cool course about graphql. For each of the requests, we can see that it was successful and it returns the list of sessions for the user. As we add more sessions and we request the favorites, we can see each one being added and returned to us. Because of the Apollo cache, all we need to do is return the favorite with the ID and the cache automatically updates to reflect the included sessions so we get our highlighted star.

Summary

So that was a good amount of work, but now we have a complete user account experience. We've secured our API and client using cookies, and we can handle page refreshes and sign-out. We used this to save and retrieve the favorite sessions for a user, and learned how to use the identity of the user to perform operations on our data. In the next module, we'll see how to expand on our knowledge and tackle authorization, which goes beyond merely knowing who the user is, and drills in on what they're allowed to do based on roles and permissions.

Restricting Access for Types and Fields with Roles and Permissions

Introduction

Now that we have our authentication in place it's time to talk about the next step in our application's evolution, authorization. We'll accomplish this by looking at how to use roles to restrict access to portions of our API. In this module we'll first take a brief look at the difference between authentication and authorization. Our user-facing interactions for saving favorites and submitting sessions works well, but we should also consider the administration side of things and how the Globomantics organization will organize their conference. For example, we may want to see what users are signed up, as well as be able to update sessions with the day they're taking place, as well as the room the session is in. We'll see how we can enable this for our Globomantics administrators and keep it out of reach for regular users while still allowing them to edit their submitted sessions both on the server side and client side. By the end of the module we'll have some basic authorization in place that will allow our Globomantics admins and our regular users' accounts to securely coexist.

Authentication vs. Authorization

Authentication and authorization can be easily confused. The previous modules have dealt with authentication, which is primarily concerned with who the user or



client of the application is. I find it easier to think of the distinction from a user's perspective. You can think of authentication as who am I. They will traditionally have a username, ID, or, in our case, an email that identifies a user and lets us, as developers, perform operations based on that information. This is usually as simple as allowed or not. This is also usually the first step in allowing a client access to an application. In contrast, authorization is what traditionally happens after authentication. You can think of authorization as what can I do, that is, once you know who the user is, you then need to establish what they're allowed to do. In the simplest situations, maybe the authorization is simply that they're logged in with no further constraints. However, in many applications, there is a need for different roles depending on who the user is or what should be allowed. An example would be akin to a traditional org chart at a company like an admin role for a VP, some organization admins that each head of department, and the employee role for everyone within those departments. In our case and for a demo for the rest of this module, we're going to establish some roles for Globomantics admins and for our users that sign up. We'll use an admin role for our Globomantics members and a user role for everyone else. We'll implement some permissions by creating a view for Globomantics admins to view all the users that have signed up and allow admins to mark speakers as featured for the conference. This will only be accessible to our signed in users based on their role, so we'll be implementing this functionality on the server-side, as well as in the React app. We'll see how to do this in our resolvers, as well as how to extract our functionality into a model within our context. We'll also see an example of using directives in our schema to establish which role is allowed to edit a type or field. Let's get into it.

Establishing Roles for Users and Admins

We need to start out by establishing our roles for admins and users. These roles will be attributes just like our email or user sub, so it will live in our JWT, just like those properties. This way, we can both verify its integrity, as well as utilize it in our client-side application. Since it's integral to our user credentials, let's begin there. We'll begin by updating our schema to add our roles. We need to have them available to return from our sign in or sign up actions along with the rest of our user data. We can define a graphql enum called role to define our admin and user roles. We'll then use this role type as part of our user type. We return this during our sign up/sign in and the user info mutations we make, so let's add that next. For our sign up mutation, we need to establish which role a user will have. For the sake of simplicity, we'll say that if their email is a Globomantics email address, they're an admin, otherwise, they're a user. We'll need to create a user record for them with the added role. And if the role is a user, we want to create a speaker record for them. We can add that in the speaker data source, it will merely take a user as an argument and create a speaker record with the users ID. This will allow us to relate a user to a speaker and any submitted sessions to



that speaker's profile. For our sign in mutation, we merely need to make sure we return the role we use when fetching the user from our database. We also need to update our user info mutation to return the role. Once we have these done, the final step is to update our createToken function to also include the role when signing our token so it's included in the claims object. With all of this out of the way, we have the server-side set up to both create the appropriate role for our users and then return that role when fetching their information or after the user signs in. All we have to do now is leverage this role when retrieving data or showing the necessary parts of the UI.

Creating the Admin Section

With our roles set up, let's quickly build the admin section of our site so we can put them to use. For the Admin section of the React app, we'll be looking to have all the user records shown for everyone that has created an account. We'll start by adding a page with some tiles for each user. I'll paste this in and we'll review it. We'll start with our users query, which will get our list of users, and for each user we'll get their favorite sessions, as well as the speaker, if there is one. We'll use the useQuery hook to grab them and iterate over them, making a little tile that shows their information, as well as links to navigate to their favorite sessions and a checkbox to mark them as a featured speaker. That checkbox will fire a mutation that will send that particular speaker's id and a featured Boolean to the server. We return the featured Boolean and user id so that the Apollo cache automatically updates and is reflected in our UI. We'll then add the Admin page to our switch in the app component. We'll use the Admin component inside our new admin route, and then import the component that shows the user list. We'll also add a new link to the header that points to our Admin page route. Once that's in place, we need to update our server to handle resolving our data and performing our mutation. We can start with the resolvers. We need to make sure that our user and speaker resolvers are set up to properly handle their sub-fields. We'll start with the user resolver. We need to add the resolver to get a speaker based on the user id. This will leverage the creation of the speaker that we added during the sign-out process. Once this is in place, we'll head to the SpeakerDataSource and add a getSpeakerByUserId method to pull the speaker from our database. Since our user now has a speaker resolver, our speaker will need a user resolver so we can retrieve the opposite relationship. This will use the userDataSource to get the associated speaker by the speaker's user id. Next we need to update our schema to add the resolver sub-fields. We'll add a user entry to our speaker type and a speaker entry to our user type. While we're here, we'll add the markFeatured mutation to our mutations type. This will take the speakerId, as well as a Boolean, for whether the speaker is featured or not. It will return the associated speaker. We'll then work our way down the stack by first adding the mutations resolver for markFeatured. This will use the speakerDataSource and pass in the two arguments for speakerId and



featured. Then we'll need to update the `dataSource` itself. We'll use the `speakerId` to find the appropriate entry in our database and then assign the `Boolean`. We'll return the result to complete our resolver. That should do it. Let's give our app a look and see our resolvers in action. We're in our brand-new Admin section and we can see our app's users. I've created a test user with an email of `mat@gmail.com` to show off our users list. You can see that the Featured Speaker checkbox shows up since this speaker was created as a user and had a speaker record created for them. If we check the Network tab, we can see the feature toggling as we mark it false, and then we can toggle it back to true and see the result return. Next, let's check out our admin role by creating a new user. We'll create one with an email of `admin@globomantics.com` to test our admin assignment logic. If we go back to the users list, we can see the user there, and this one doesn't have a Featured Speaker checkbox because it doesn't have a speaker associated with it. If we open the Network tab and refresh the page, we can see that `userInfo` call that comes back to confirm that our user role is indeed admin. We have everything in place now to authorize our users and admins based on these roles. Next, we'll take a look at how to leverage them in our resolvers.

Authorization in a Resolver

Authorizing within resolvers is straightforward. We already have access to our user, so let's see what it looks like. We'll go through a couple examples starting with our `markFeatured` mutation. We want to restrict this to only admin users, so inside our mutation file at our `markFeatured` resolver, we can start by destructuring the user and `dataSources` from the context. We'll update our call to use just the `dataSources`, and then wrap the whole thing in a check for our user, as well as a check to see if the role is equal to `ADMIN`. If this isn't the case, we'll return null. Yep, simple as that. Let's see it in the app. In our application, we're logged in as the regular test user, and we can see our checkbox for the Featured Speaker, but if we click it, nothing happens. If we check the Network tab, we can indeed see that the result is coming back null, as we're not actually performing the mutation. The fact that we've locked this down is helpful, but we really don't want regular users to be able to interact with the user list at all, or even see the page to begin with. Let's fix that. We'll head to our users resolver within `query.js` and make the same changes by destructuring the user and `dataSource` and checking for the user and admin role before making the call to `dataSources`. If this isn't the case, we'll go ahead and return an empty array. We could also return null or even throw an error in the case to let the client know that this isn't allowed. Back in the app, if we visit the page again, we'll notice that nothing shows up. There are no users. We can confirm this by looking at the `graphql` call in the Network tab and seeing our empty array, since our test user is not an admin. The final step will be to restrict access to our UI. We'll start in the `AuthProvider` by making a new value for `isAdmin` that checks if our user role is admin. We add that to the context value so it's available in our application. Next,



we'll modify our `app.jsx` file to pull in the context, and we'll destructure our `isAdmin` flag from it. We'll wrap our route in a check to see if it should show up when the app loads. For even more security, you could lazy load the admin component so the JavaScript isn't loaded at all. Now let's see it in the app. We won't see anything on the Admin page since we're signed in as a regular user. We can sign in as our admin user, head back to the page, and we'll see our user list. Next, we'll look at how to abstract our authorization logic into a model.

Authorization in a Model

Extracting authorization logic into a separate model layer as opposed to it being in the resolvers themselves is a good best practice to adopt as your API grows. Let's see how we might accomplish this in our API. When we think about authorization with GraphQL, we need to also keep in mind that our types can be resolved in many different ways and in many different places. Types can be nested within other types, and can therefore be resolved in top-level queries and as resolvers within those queries. In our app, we have a top-level query for getting a user by their id. If an admin is authorized, we want the query to succeed. If a regular user is authorized, we only want them to be able to grab their own id. Unfortunately, we have the same logic in multiple places in our app. It's exposed as the `me` query that we use to allow a user to get their list of favorite sessions in the app, and it's also within the `speaker`'s resolver so we can get the user record associated with a particular speaker. If we wanted to authorize these calls appropriately, we don't want to have the same checks performed in each of the resolvers. For this problem, the solution is using a model. Now the name model is not special, you can call it whatever you like. The concept is that there is a layer between your resolvers and whatever is fetching your data that will be able to apply your authorization rules in a uniform way. The resolver is merely the router for your queries. The model applies the authorization or business logic and the datasource or other functions fetch the data. This way, no matter where you're resolving data, the authorization rules are the same in each case. Let's walk through updating our `userById` resolver to use a model. We'll start by creating the model itself. We'll make a new folder to hold our models, and inside we'll create `user.js`. We'll first import a `getUserById` function from our user datasources file. We'll create this function in a minute. We'll declare a function called `generateUserModel` that takes an object containing a user. This will be the user passed in off the context. This function is going to use the user to create an object containing methods that will contact our database. We're going to check the role for the user. If it's admin, we'll return based on the id that was passed in. Otherwise, we'll use the `user.sub` value to allow users to only retrieve themselves. We'll export this, and then we can use it in our server declaration. Let's head to the server file and import our `generateUserModel` method. Now that that's done, we'll use this in our context function. We want to expose the model on our context so we can use it whenever we want within the



resolvers or datasources. We'll create a models object that will hold a model we'll call user, which will be the result of calling our generateUserModel with our user created from our verified token. Next, we'll head to our datasource to utilize our model. To start with, we're going to extract the database into a constant. We'll call it userDb. At this point, we also want to export the function that will return the id using this newly declared database. This is going to be the function that is used by our generateUserModel function. At this point, we will need to import our model from context. We can do this by adding another entry in the initialize function. This gives us access to a config object that contains the context and will allow us to assign a new model variable. All we need to do next is use our model. We can replace the call in the getUserById method with our model variant. The last step is to add it to our exports. We'll head back to the server file and update our import because we're now exporting an object from the user datasource file. Alright, we should be good. Let's see it in action. Back in the app, let's start off by signing in as our admin user. We won't be using the actual application to test out our new authorization methods, we'll be using the GraphQL Playground. Since we're using cookies as our authentication method, we need to go to our Settings and make sure that the request.credentials value is set to include. If you don't have this, go ahead and add it. Back in our users tab, we can run the request and get our list of users. We'll copy off the ids for each one because we'll use them in our userById queries to test our authorization. Next we'll implement each of them. We'll start with the admin. We'll alias the userById query to admin, and paste in the admin's id. We'll follow that by adding the user query. We'll alias this to user and update the id to reflect the user id. We can run our query and see our results come back. This is expected, since our admin user is authorized to return any user by their id. Next, we'll test out our user authorization role. We'll head back to the app and sign in as our regular user account. We'll go back to the Playground and run the query again. We get much different results this time because our user role is only authorized to return itself, so for the admin and the user we get the same actual user results back. So this is pretty great. We've abstracted away our authorization logic into a model so that the userById query can be handled exactly the same way in all of our resolvers.

Authorization Using Schema Directives

The last thing we'll look at is how to implement simple authorization rules using schema directives. To do this, we'll again take a look at our users query. We only want to allow admins to request a list of users, but right now we have that check in the resolver. We can clean up our resolver by removing that check and move the logic into a new directive that we can use to annotate our schema.

By annotating specific fields, we'll be able to restrict access to them and ensure that only admins can request them. Our directive is going to be fairly simple, we'll merely use it when we want to require that the user accessing the field has an admin role. To get started with our directive, we first want to declare it in our



schema, we'll call it `requiresAdmin`. Next, let's implement our directive. We'll make a new folder in the root of the project called `directives`, and in that, we'll create an `AuthDirective.js` file. We'll start by importing the `SchemaDirectiveVisitor` class and the `AuthenticationError`. We're going to extend this class and implement the `visitFieldDefinition` method. We're implementing this method because we declared our directive to be on a field definition. The signature gives us access to the field and some details. We'll only worry about the field for now. We're going to grab the context, which is available on the args of the function, and pull the user off so we can check our role. If the role is not admin, we want to throw an authentication error so we can inform the client. If it is, we'll resolve the field normally. Once this is exported, we can use it in our server. Back in our `server.js` file, the next step is to set up the `AuthDirective` that we just created. We can add a schema directive entry to our server config. The key will match the `requiresAdmin` directive we declared, and the value will be our newly defined `AuthDirective` class. We'll make sure to import it at the top. And now that everything is in place, we'll add our directive to our users entry. Now that it's done, let's try it out in the playground. Back in the application, we'll start by logging in as our admin user. When we go to the graphql playground, we can run the query for the users list. This comes back with our users as expected. Now let's try the regular user account. We'll log in as our user and head back to the playground. When we run our query, we can see the authentication error that we added come back in the response. It works.

Summary

Alright, our authorization is complete. We have successfully created admin and user roles for our application, and leveraged them in multiple ways. The user and admin roles we created allows our application to service the needs of our users and Globomantics administration in the same application. These could be expanded even further in your own applications to match any project use case. Using authorization and resolvers gives us a simple way to make sure that only the specific roles we want are allowed to access certain resources. Abstracting away authorization logic to a model lets us create reusable functions to enforce our authorization rules in all the places that a type is resolved, while at the same time simplifying our actual resolvers. Using schema directives to apply authorization lets us abstract out rules in a different way, keeping them more readable, while allowing us to apply them to any type or field we want. In the next module, we're going to look at how to handle large and complex queries, and although not related directly to authentication or authorization, it's an important step in protecting against misuse or malicious actors.

Protecting Against Large and Malicious



Queries

Introduction

Our security concerns so far have been related to the users and admins of our application. For this module, we're going to look at some of the inherent issues with GraphQL in regards to arbitrary client queries, the issues surrounding them, and how to protect against them. We've got our server set up to authenticate and authorize our users so we know who they are and what they're allowed to do. At this point, authorized users can make whatever requests they want as long as the queries or mutations themselves are allowed. Although we want to be as accommodating as possible, this can still pose some risks beyond just authentication and authorization. GraphQL allows clients to create arbitrary queries to get whatever they want. This is a great benefit from the perspective of a client. However, these queries could end up causing issues with our server, whether intended or not. These potential issues include, among others, clients performing many queries or mutations within a short amount of time, clients performing very deeply nested queries, and clients performing complex queries. By preventing these issues where possible, we can lessen the risk of our server performing badly or even being taken down intentionally by malicious actors who would use our queries to cause extensive resource utilization. For a demo, we're going to explore some possible solutions for these issues and tackle three approaches. We'll start by looking at how to add rate-limiting to our queries so we can prevent multiple queries from being fired within a certain amount of time. We'll then look at how to prevent excessively nested queries from being executed, by applying a check for query depth. Finally, we'll see how to add some query complexity checks that would allow the API to reject overly complex queries. By the end of the module, we'll have a few basic strategies implemented for further securing our API.

Implementing Rate-limit Protection

Let's look at how to implement rate-limit protection. Rate-limiting is a simple first step to prevent an excessive number of requests from being sent all at once, possibly creating a Denial of Service for clients of your API. We'll implement this particular solution by installing a directive provided by a library called `graphql-rate-limit`. In the API directory of the project, we'll run `npm install graphql-rate-limit-directive`. We'll head to the server file and add an import for it at the top. We'll pull in a few functions, specifically, `createRateLimitTypeDef`, `createRateLimitDirective`, and `defaultKeyGenerator`. We'll begin by adding a `schemaDirective` entry in our server config. We'll call it `rateLimit`, and use the imported `createRateLimitDirective` function. We'll also need to add the type definition for the rate limit. We'll add this to our `typeDefs` in the server



config. Next, we can head to our schema file. We're going to add our `RateLimitDirective` to our `Query` type. As this is a simple API and we want to try out the functionality, we'll set the values for this directive fairly low. We'll set the limit at 5 and the duration at 10. This will limit any queries from happening more than 5 times in 10 seconds. This will work for simple queries. The way we have it set now will limit all queries for all clients to the same rate, but since we know who the user is, we can make this a bit smarter. We can use the fact that we know who the user is, at least for some queries, to rate-limit on a per-user basis instead of using one blanket rule. Back in the server file, we can add a key generator to our `RateLimitDirective` as an option. By creating this function, we can tell the `rate-limit-directive` library how to differentiate different requests so they can be rate limited appropriately. It takes a number of parameters, one of which being `context`. We can leverage the `context` to see if we have a user. If we do, we'll use the `user.sub` value, their id, to create a string key while also using the `defaultKeyGenerator` provided by the library. If we don't have a user, we'll just use the `defaultKeyGenerator` itself. That's all we need for our rate-limit implementation. Let's try it out in the Playground. Let's start with a simple query for sessions. We'll grab the id and title for each one and hit the Play button. We can see our sessions return, so that's good. I'll hit it another five times and see the error message return, telling us to wait a bit longer. Great, that works too. This solution can be applied to any other types or fields you want to specifically limit, especially if each call results in an expensive lookup or other operation.

Implementing Depth-limit Protection

Now let's look at how we can protect against nested queries by implementing a depth limit. Nested queries are a great way to grab all the data you may need in one request. However, many times you may have types that are related to one another. In our case, we have sessions, speakers, and users that are all related to each other. If we request sessions, we can then request the speakers for each of those sessions, we can then request the users tied to each speaker, and we could even request the sessions again for each speaker. Granted, this is a bit contrived. With a more complex schema, this could easily cause many, many requests and different resolvers to fire. Maybe that's what you want to allow, which is fine, however, someone could also use the same method to nest speakers in sessions and speakers and sessions on and on causing massive resource utilization. This could prove to be very expensive from a hosting standpoint and perhaps even prevent your users from accessing the API. We can prevent this by adding a depth limit to queries and our server configuration. We can start by adding the `graphql-depth-limit` package to our application. Again, in the `API` folder, we'll run `npm install graphql-depth-limit`. Towards the top of the server file, we can add the import for the library. In our server config, we'll add a new entry called `validation rules` that will let us pass an array of functions. In this case, we can call `depthLimit` with a value to limit how many levels of nesting will be allowed in our



server. We'll use 3 just to show how this works, but in a real scenario, you could likely get away with a higher value. That's all we need for this one. Let's see our queries now that we have this in place. Back in the playground, we have the same query that we started with. If we run it, we can see that our error does indeed show up letting us know the query had too much nesting. Let's remove the extra session and speakers on the end there and try it again. Alright, we can see our data again, so we're good. With this restriction in place, we can make sure nested queries can't bring our server to a halt.

Implementing Protection Against Query Complexity

The final issue we'll look at is query complexity. Query complexity is a way to quantify the amount of time or resources queries can use. The basic idea is that every requested scalar, object or list can be valued at a certain numerical amount. When a query is performed, all those values are added up, and that value is the complexity score. This could also be thought of as a cost to run the query. This goes beyond merely seeing if a query is nested a certain amount. These values can vary based on how they're resolved, so these scores can be curated and tuned to match any specific server implementation fairly well. If we know that certain fields or types will be more expensive to resolve, we can encode it in our configuration so that it's reflected accurately. To see it in action, let's install a library that will handle these calculations. Once again, we'll head to the API directory. We can run `npm install graphql-validation-complexity` to add it to our project. This will allow us to configure some nice default values and functions for marking certain operations as complex, and allow us to restrict appropriately expensive queries from being executed. Towards the top of the server file, let's import it. In our Apollo server config, let's add a simple validation rule to the array we established previously. We'll set a basic rule that restricts our server from performing queries with a complexity above 600. We can also use a callback to watch for when the cost is calculated, and in our case, we'll just log it to the console so that we can monitor our queries. I'll open up GraphQL Playground next to our window so we can see our query cost logging in action. We'll start with a basic query for a sessionById. This one looks cool. We can see that it has a fairly low score. Scalar values are only counted as one each by default, so we're in good shape. Let's add a few more values. We can see once we run the query that the score goes up. Next, we can add a speaker entry to the query. If we add a few scalar values in there, we can see the score jump up by 30. The factor used for lists is 10, so every scalar we add in a nested list will increase the value by 10. Extrapolating this out could give us a higher score for every nested relation we add or value we want to return. Let's look at another example of configuring the cost of our queries. This time we'll look at the schema. Our library also makes a cost directive available to us, so let's add that to the schema. In this hypothetical example, let's say that our description field is actually pulled from a remote REST endpoint that uses a GPT-3 implementation



to generate a description using an advanced neural network. That sounds expensive. In this case, let's apply our cost directive to the description field. Bumping this value up will increase our total cost substantially. In this case, it's enough to push us over the limit we placed above. We can see the error it returns, letting us know that the current query is too complex. This is helpful because the query is otherwise not very complicated. It's not even very deep. We could apply different rules as needed to different values or types in our schema, or in the configuration itself, to align the complexity score with the cost of resolving our different types. You can leverage this same approach in your projects where it suits the situation.

Summary

So that's it. We now have a solid basis for applying solutions to common issues that are inherent to the nature of graphql queries. Hopefully, these allow you to create a secure foundation for your own APIs. For a more in-depth look at performance, specifically, check out the Apollo performance management playbook [here](#), on Pluralsight. Rate-limiting, depth-limiting, and complexity protections go a step beyond more basic measures, such as pagination or limiting the size of your responses, in general. As we've seen, the benefits are not difficult to obtain once the right set of tools is in place. If you made it this far, cheers to you. I hope this module and this course, in general, has given you a better understanding of what makes security practices like these important, as well as how to apply it effectively in your own applications. Thank you for watching.